

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the modern digital era, data plays a crucial role in decision-making processes across various domains. Organizations and individuals generate large volumes of data daily through different applications and systems. However, raw data alone is not useful unless it is properly processed, analyzed, and converted into meaningful insights.

The proposed system, User-Centric Decision Intelligence System, is designed to integrate user interaction with data-driven analytics. It combines two major components: a user-based system (Expense Tracker module) and a decision intelligence module that analyses datasets to generate insights. This integration enables users to not only manage personal data but also derive meaningful conclusions from structured datasets.

The system is developed using Python and web-based technologies, ensuring simplicity, flexibility, and ease of use. It aims to bridge the gap between user input systems and analytical decision-making systems by providing an interactive and intelligent platform.

1.2 Problem Statement

Traditional systems either focus on data entry applications (such as expense trackers) or separate analytical tools for data processing. These systems operate independently, making it difficult for users to connect personal data management with analytical decision-making.

There is a need for a unified system that:

- Allows users to input and manage personal data efficiently
- Processes external datasets for analysis
- Provides meaningful insights for better decision-making

The lack of integration between user data systems and analytical intelligence systems motivates the development of this project.

1.3 Scope and Objectives

Scope

The scope of this project includes:

- Development of a user-friendly expense tracking system
- Implementation of dataset-based analysis for decision-making
- Integration of both systems into a single platform
- Generation of insights based on structured data inputs

The system is designed as a software-based application and does not require any specialized hardware.

Objectives

The main objectives of the system are:

- To design a simple and interactive user data management system
- To perform analysis on given datasets for extracting useful insights
- To integrate user input data with analytical processing
- To provide meaningful visual or textual outputs for decision support
- To improve user understanding of data-driven decision-making

1.4 Methodology

The system follows a structured development approach combining both user interface design and data analysis techniques.

The methodology includes the following steps:

1. User Data Collection

- Users input data through the Expense Tracker module.

2. Data Storage and Management

- The system stores and organizes user inputs for future processing.

3. Dataset Integration

- External datasets are loaded into the system for analysis.

4. Data Processing and Analysis

- Python-based logic is used to clean, process, and analyze data.

5. Insight Generation

- The system generates outputs such as summaries, trends, or patterns.

6. User Interaction

- Users can view results in an understandable format.

This structured workflow ensures smooth integration between user input and decision intelligence.

1.5 Project Overview

The User-Centric Decision Intelligence System is divided into two main modules:

1. Expense Tracker Module (User Integration)

This module allows users to record and manage their daily expenses. It focuses on user interaction, data entry, and simple tracking of financial activities.

2. Decision Intelligence Module

This module processes datasets using Python to generate analytical insights. It helps in identifying patterns, trends, and meaningful conclusions from structured data.

By combining these two modules, the system provides both personal data management and analytical decision support, making it a hybrid intelligent system.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction to Literature Survey

A literature survey is an essential phase in system development that involves studying existing systems, research works, and applications relevant to the proposed project. It helps in understanding current technologies, identifying gaps, and defining improvements required in the new system.

For the User-Centric Decision Intelligence System, the literature survey focuses on two key domains:

- Personal finance management systems (Expense tracking applications)
- Data analytics and decision intelligence systems

This dual-domain approach is required because the proposed system integrates both user-level data entry and analytical decision-making capabilities.

2.2 Study of Existing Systems

A. Expense Tracking Applications

Modern expense tracking applications such as mobile-based budgeting tools and web-based financial trackers allow users to record income and expenses.

Key features observed:

- Manual entry of transactions
- Category-based expense classification
- Monthly summary reports
- Basic visualization (charts, graphs)

Limitations:

- Limited intelligence (no predictive analysis)
- No integration with external datasets
- No decision support capabilities
- Mostly static reporting systems

B. Data Analytics Systems

Data analytics platforms such as Excel dashboards, Python-based Pandas analysis tools, and BI systems (like Power BI and Tableau) are widely used for data interpretation.

Key features observed:

- Data cleaning and preprocessing
- Statistical analysis and visualization
- Pattern recognition in datasets
- Dashboard-based insights

Limitations:

- Lack of user-level interaction systems
- Requires technical knowledge for usage
- Not integrated with real-time user-generated data
- Often disconnected from personal applications

C. Hybrid Intelligent Systems

Recent advancements in 2023–2026 have introduced systems that combine user interaction with analytics, often referred to as decision intelligence platforms.

Examples:

- AI-based financial assistants
- Smart dashboards with user personalization
- Automated insight generation systems

Limitations:

- Complex implementation
- Often cloud-dependent
- Limited customization for academic/simple user systems
- Not designed as unified beginner-friendly systems

CHAPTER 3: SYSTEM REQUIREMENT SPECIFICATION

3.1 Introduction

The System Requirement Specification (SRS) defines the functional, software, and system-level requirements of the User-Centric Decision Intelligence System. The system is designed as a web-based intelligent platform integrating user interaction modules with data analytics capabilities.

The application is developed using HTML, CSS, JavaScript for the frontend and Python for backend processing, along with VS Code as the development environment. The system enables users to input financial data and analyzes datasets to generate meaningful decision-oriented insights.

3.2 System Architecture Requirements

The system follows a layered web-based architecture consisting of:

- **Frontend Layer:** User interface built using HTML, CSS, and JavaScript
- **Backend Layer:** Python-based processing logic
- **Data Layer:** CSV/JSON-based storage for user and dataset information
- **Analytics Layer:** Python modules for data processing and decision intelligence

3.3 Software Requirements

The system is implemented using the following software stack:

Frontend Technologies:

- HTML5 – Structure of web pages
- CSS3 – Styling and layout design
- JavaScript – Client-side interactivity

Backend Technologies:

- Python 3.x – Core processing and analytics engine

Tools:

- Visual Studio Code – Development environment
- Browser (Chrome/Edge) – For testing UI

Libraries (Python):

- Pandas – Data manipulation
- NumPy – Numerical processing
- Matplotlib (optional) – Data visualization
-

3.4 Assumptions

The system is designed with the following assumptions:

- Users have basic knowledge of operating a web interface
- Input data provided by users is valid and structured
- Dataset used for analysis is pre-cleaned or partially structured
- System runs in a local development environment (VS Code)

CHAPTER 4: SYSTEM ANALYSIS

4.1 Introduction

System analysis involves evaluating existing solutions and identifying their limitations to justify the development of a new system. This chapter examines current approaches in expense management and data analytics, and highlights the need for an integrated solution.

The **User-Centric Decision Intelligence System** is designed to bridge the gap between user data management and intelligent data analysis by combining both functionalities into a single platform.

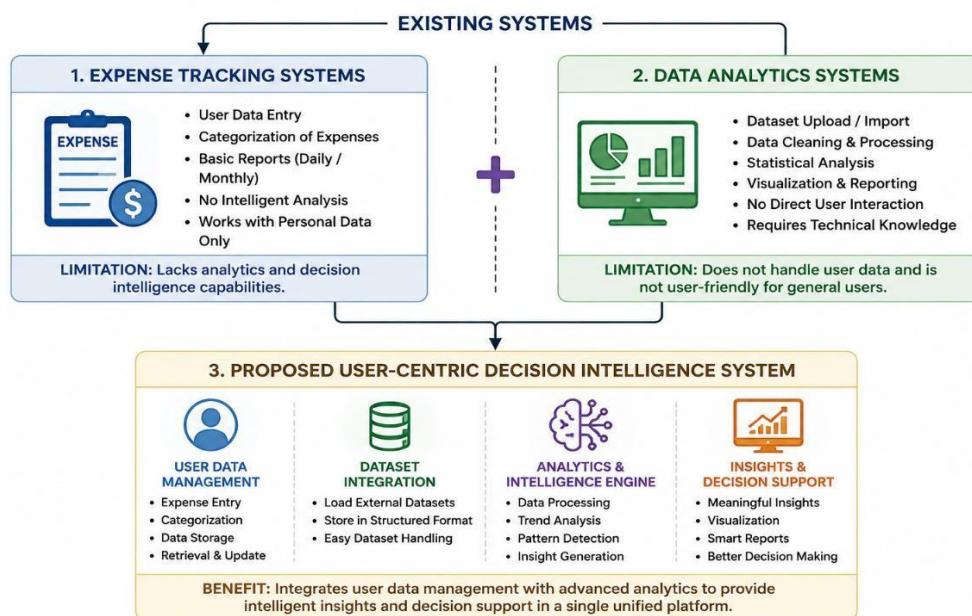


Figure 4.1.1: Conceptual Representation of Existing vs Proposed System

4.2 Comparative Analysis

- A comparison between existing systems and the proposed system is presented below:

Feature	Expense Tracking Systems	Data Analytics Systems	Proposed System
User Data Management	Supports basic personal data entry	Does not support user data management	Supports complete user data handling
Dataset Analysis	Not supported	Fully supported	Fully supported
Decision Intelligence	Not available	Limited analytical insights	Provides meaningful decision insights
User-Friendly Interface	Simple and user-friendly	Requires technical knowledge	Designed for ease of use
System Integration	Operates independently	Operates independently	Integrates both functionalities

Table 4.3.1: Comparative Analysis of Existing and Proposed Systems

4.3 Proposed System

The proposed system integrates user interaction with data analytics into a unified platform.

Key Characteristics:

- Web-based interface using HTML, CSS, and JavaScript
- Python-based backend for processing
- Modular architecture separating user and analytics components
- Local data storage using CSV/JSON

CHAPTER 5: SYSTEM DESIGN

5.1 Introduction

System design defines the structure and working of the User-Centric Decision Intelligence System. It describes how different components of the system interact to perform data upload, processing, and result generation.

The system is designed as a web-based application with a clear separation between frontend interface, backend processing, and data storage. The design ensures smooth user interaction, efficient data handling, and accurate generation of analytical insights.

5.2 System Architecture

The system follows a layered architecture consisting of:

- Frontend Layer: Developed using HTML, CSS, and JavaScript to provide user interaction
- Backend Layer: Implemented using Python for processing and analysis
- Data Storage Layer: Stores uploaded datasets and maintains history
- Analysis Layer: Performs data processing and generates insights

The architecture ensures that user actions from the interface are processed through the backend and results are displayed effectively.

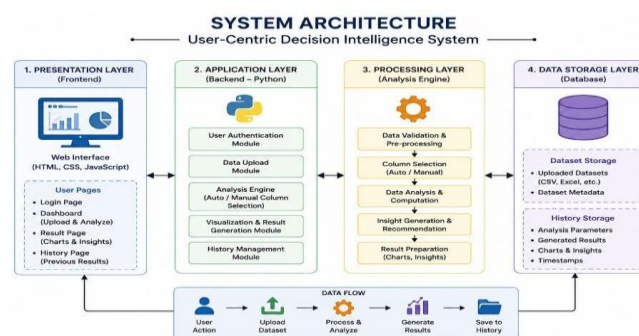


Figure:5.2.1 Systematic Architecture

5.3 System Flow

The system follows a structured workflow starting from user authentication to result generation.

Flow Description:

- The user logs into the system
- The dataset is uploaded through the dashboard
- Uploaded data is stored and recorded in history
- The user initiates analysis from the same dashboard
- The system prompts for column selection (automatic or manual)
- Data is processed using backend logic
- Results including charts, insights, and recommendations are displayed
- Users can also access previous results through the history feature

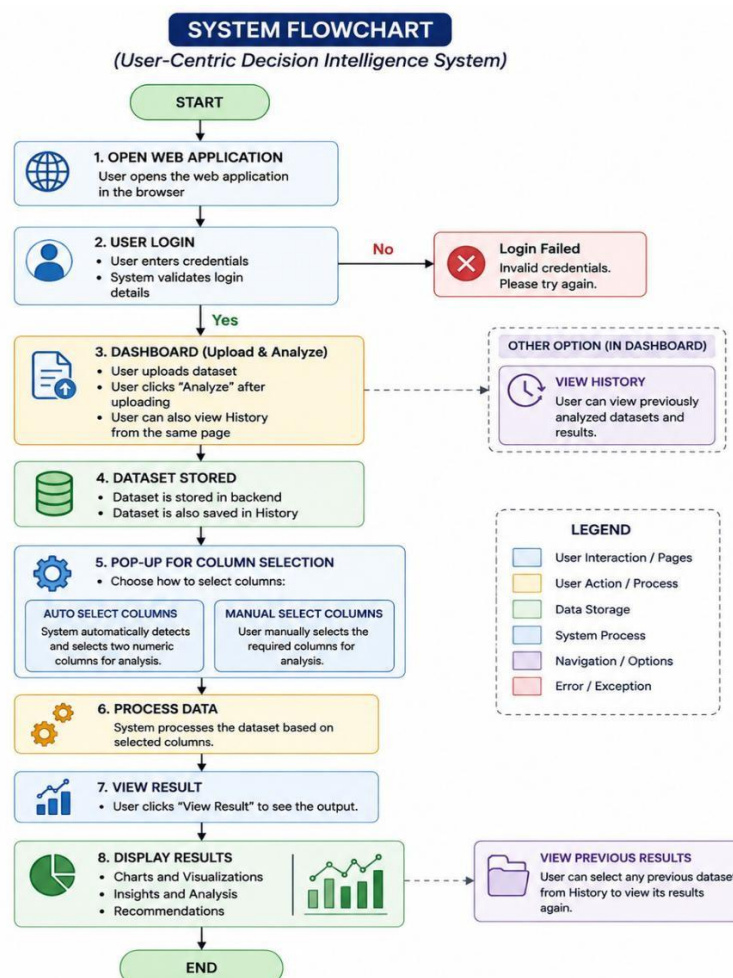


Figure 5.4.1: Flowchart of the Proposed System

5.4 Data Design

The system uses structured data formats for storage and processing.

Dataset Structure:

- Tabular format (CSV or similar)
- Contains multiple columns including numerical and categorical data

Stored Information:

- Uploaded dataset details
- Selected columns for analysis
- Generated results and insights

Field Name	Description
Dataset Name	Name of uploaded file
Columns	Attributes present in dataset
Selected Data	Columns used for analysis
Output	Generated results and insights

Table 5.5.1: Data Structure Overview

CHAPTER 6: SYSTEM IMPLEMENTATION

6.1 CODE IMPLEMENTATION

```
from flask import (
    Flask,
    render_template,
    request,
    jsonify,
    session
)

from flask_cors import CORS
import pandas as pd
import os
from datetime import datetime

app = Flask(__name__)

app.secret_key = "decision_intelligence"

CORS(app)

UPLOAD_FOLDER = "uploads"

os.makedirs(
    UPLOAD_FOLDER,
    exist_ok=True
)
```

Figure 6.1.1

```
@app.route(
    "/analyze",
    methods=["POST"]
)
def analyze():
    global current_analysis

    if "file" not in request.files:
        return jsonify({
            "error":
                "No file uploaded"
        })

    file = request.files["file"]

    mode = request.form.get(
        "mode",
        "auto"
    )

    x_axis = request.form.get(
        "x_axis"
    )

    y_axis = request.form.get(
        "y_axis"
    )

    filename = file.filename

    path = os.path.join(
        UPLOAD_FOLDER,
        filename
    )
```

Figure 6.1.2

```
try:

    if filename.endswith(
        ".csv"
    ):

        df = pd.read_csv(
            filepath
        )

    elif filename.endswith(
        ".xlsx"
    ):

        df = pd.read_excel(
            filepath
        )

    else:

        return jsonify({
            "error":
                "Unsupported file"
        }), 400

except Exception as e:

    return jsonify({
        "error":
            str(e)
    }), 500

columns = list(df.columns)

numeric_columns = list(
    df.select_dtypes(
        include="number"
    ).columns
)

text_columns = [
    col for col in columns
    if col not in
```

Figure 6.1.3

```

from services.history_service import save_analysis_history
from services.narrative_service import generate_narrative
import pandas as pd

from services.visualization_service import generate_bar_chart

def run_auto_analysis(filepath):

    extension = filepath.rsplit(".", 1)[1].lower()

    # CSV FILE
    if extension == "csv":

        df = pd.read_csv(filepath)

    # EXCEL FILE
    else:

        df = pd.read_excel(
            filepath,
            engine="openpyxl"
        )

    # Detect numeric columns
    numeric_columns = (
        df.select_dtypes(include=["number"])
        .columns
        .tolist()
    )

```

Figure 6.1.4

```

# Select columns automatically
y_column = numeric_columns[0]

# Choose first non-numeric column for x-axis
non_numeric_columns = (
    df.select_dtypes(exclude=["number"])
    .columns
    .tolist()
)

if len(non_numeric_columns) > 0:

    x_column = non_numeric_columns[0]

else:

    x_column = df.columns[0]

# Generate chart
chart_json = generate_bar_chart(
    df,
    x_column,
    y_column
)

# Generate narrative
narrative = generate_narrative(
    df,
    x_column,
    y_column
)

```

Figure 6.1.5

```
from services.history_service import save_analysis_history
from services.narrative_service import generate_narrative
import pandas as pd

from services.visualization_service import generate_bar_chart

def run_manual_analysis(
    filepath,
    sheet_name,
    x_column,
    y_column
):
    extension = filepath.rsplit(".", 1)[1].lower()

    # CSV FILE
    if extension == "csv":
        df = pd.read_csv(filepath)

    # EXCEL FILE
    else:
        df = pd.read_excel(
            filepath,
            sheet_name=sheet_name,
            engine="openpyxl"
        )
```

Figure 6.1.6

```
# Validate x column
if x_column not in df.columns:
    raise Exception(
        f"{x_column} column not found."
    )

# Validate y column
if y_column not in df.columns:
    raise Exception(
        f"{y_column} column not found."
    )

# Generate chart
chart_json = generate_bar_chart(
    df,
    x_column,
    y_column
)

# Generate narrative
narrative = generate_narrative(
    df,
    x_column,
    y_column
)
```

Figure 6.1.7


```
def generate_narrative(df, x_column, y_column):  
  
    # Highest value row  
    highest_row = df.loc[df[y_column].idxmax()]  
  
    # Lowest value row  
    lowest_row = df.loc[df[y_column].idxmin()]  
  
    # Average value  
    average_value = round(df[y_column].mean(), 2)  
  
    # Total value  
    total_value = round(df[y_column].sum(), 2)
```

Figure 6.1.8

```
# Insights  
insights = (  
    f"The dataset contains a total value of "  
    f"{total_value} across all records. "  
    f"The average {y_column} is {average_value}. "  
    f"The highest contribution comes from "  
    f"{highest_row[x_column]} with a value of "  
    f"{highest_row[y_column]}. "  
    f"The lowest contribution comes from "  
    f"{lowest_row[x_column]} with a value of "  
    f"{lowest_row[y_column]}."  
)  
  
# Analysis  
analysis = (  
    f"The analysis indicates that "  
    f"{highest_row[x_column]} dominates the dataset "  
    f"compared to other categories. "  
    f"There is a noticeable variation between "  
    f"the highest and lowest values, suggesting "  
    f"an uneven distribution pattern in the data."  
)  
  
# Recommendation  
recommendation = (  
    f"It is recommended to closely monitor "  
    f"{highest_row[x_column]} since it represents "  
    f"the largest share of the dataset. "  
    f"Optimizing or balancing the distribution "  
    f"may improve overall efficiency and stability."  
)
```

Figure 6.1.9

```
import plotly.express as px

def generate_bar_chart(df, x_column, y_column):

    figure = px.bar(
        df,
        x=x_column,
        y=y_column,
        title=f"{y_column} by {x_column}"
    )

    return figure.to_json()
```

Figure 6.1.10

```
import json

from flask import Blueprint, jsonify

history_bp = Blueprint("history_bp", __name__)

HISTORY_FILE = "history/analysis_history.json"

@history_bp.route("/history", methods=["GET"])
def get_history():

    try:

        with open(HISTORY_FILE, "r") as file:

            history = json.load(file)

            return jsonify({
                "status": "success",
                "data": history
            })

    except Exception as error:

        return jsonify({
            "status": "error",
            "message": str(error)
        }), 500
```

Figure 6.1.11

```
from flask import Blueprint, request, jsonify

from services.auto_analysis_service import run_auto_analysis
from services.manual_analysis_service import run_manual_analysis

analyze_bp = Blueprint("analyze_bp", __name__)

@analyze_bp.route("/analyze", methods=["POST"])
def analyze_file():

    try:

        data = request.get_json()

        filepath = data.get("filepath")
        mode = data.get("mode")

        # AUTO MODE
        if mode == "auto":

            result = run_auto_analysis(filepath)

        # MANUAL MODE
        elif mode == "manual":

            result = run_manual_analysis(
                filepath=filepath,
                sheet_name=data.get("sheet_name"),
                x_column=data.get("x_column"),
                y_column=data.get("y_column")
            )

        else:

            return jsonify({
                "status": "error",
                "message": "Invalid mode selected."
            }), 400

        return jsonify({
            "status": "success",
            "data": result
        })

    except Exception as error:

        return jsonify({
            "status": "error",
            "message": str(error)
        }), 500
```

Figure 6.1.13

```
from flask import Blueprint, request, jsonify

from services.file_service import save_uploaded_file

upload_bp = Blueprint("upload_bp", __name__)

@upload_bp.route("/upload", methods=["POST"])
def upload_file():

    # Check file exists
    if "file" not in request.files:
        return jsonify({
            "status": "error",
            "message": "No file provided"
        }), 400

    file = request.files["file"]

    # Check filename
    if file.filename == "":
        return jsonify({
            "status": "error",
            "message": "Empty filename"
        }), 400

    try:
        result = save_uploaded_file(file)

        return jsonify({
            "status": "success",
            "data": result
        })

    except Exception as error:

        return jsonify({
            "status": "error",
            "message": str(error)
        }), 500
```

Figure 6.1.14

```
const chartType =  
document.getElementById(  
"chartType"  
);  
  
const backBtn =  
document.getElementById(  
"backBtn"  
);  
  
const insightText =  
document.getElementById(  
"insightText"  
);  
  
const recommendationText =  
document.getElementById(  
"recommendationText"  
);  
  
const analysisText =  
document.getElementById(  
"analysisText"  
);  
  
let chart;
```

Figure 6.1.15

```
async function loadChart(  
type = "bar"  
) {  
  
    const res =  
        await fetch(  
            "/get-results"  
        );  
  
    const data =  
        await res.json();  
  
    const ctx =  
        document.getElementById(  
            "resultChart"  
        );  
  
    if (chart)  
        chart.destroy();  
  
    chart =  
        new Chart(  
            ctx,  
            {  
                type: type,  
  
                data: {  
                    labels:  
                        data.labels,  
  
                    datasets: [{  
                        label:  
                            data.y_axis,  
  
                        data:  
                            data.values,  
  
                        borderWidth: 1  
                    }]  
                }  
            },  
        );  
}
```

Figure 6.1.16

```
function createNarrative(  
  data  
) {  
  
  const values =  
    data.values;  
  
  const total =  
    values.reduce(  
      (a,b)=>a+b,  
      0  
    );  
  
  const avg =  
    (  
      total /  
      values.length  
    ).toFixed(2);  
  
  const max =  
    Math.max(  
      ...values  
    );  
  
  const min =  
    Math.min(  
      ...values  
    );  
  
  const highest =  
    data.labels[  
      values.indexOf(max)  
    ];  
  
  const lowest =  
    data.labels[  
      values.indexOf(min)  
    ];  
}
```

Figure 6.1.17

6.2 WEBSITE DESIGN

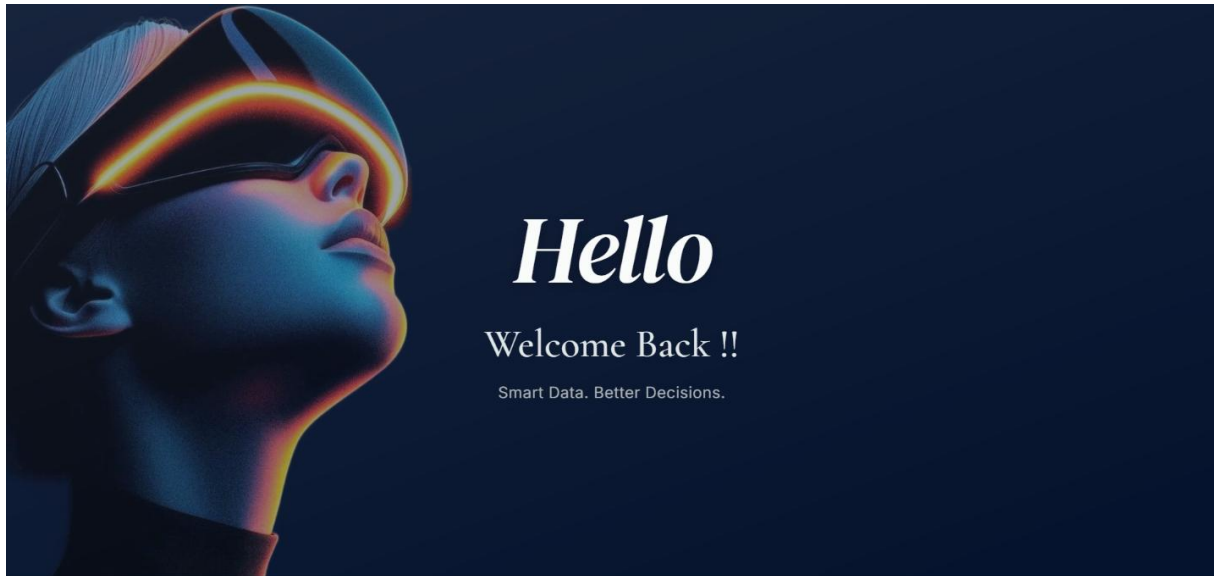


Figure 6.2.1

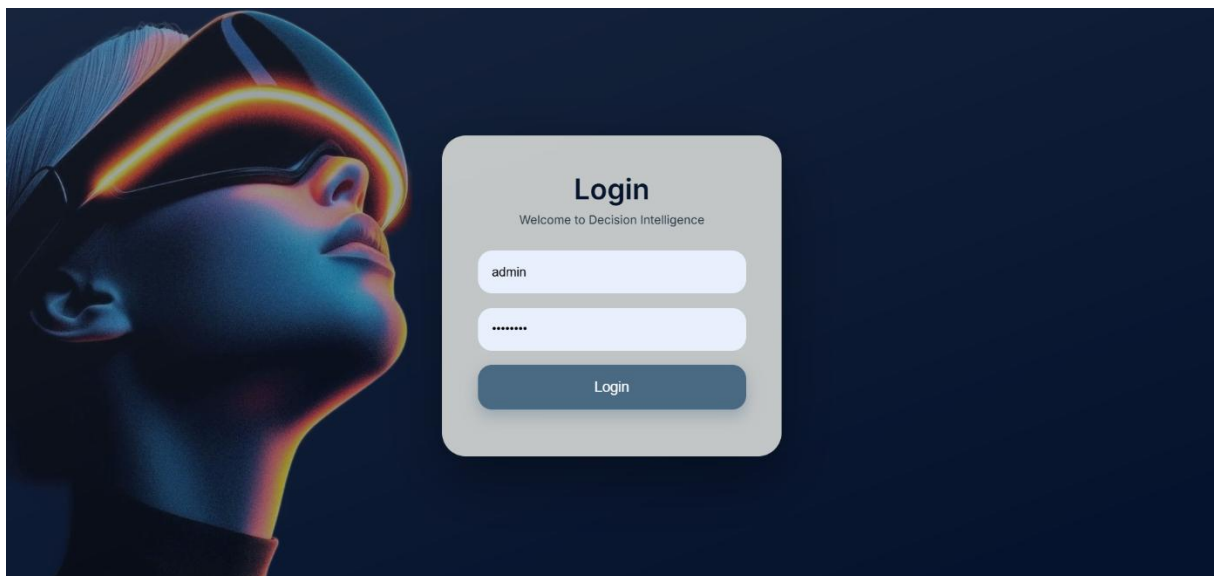


Figure 6.2.2

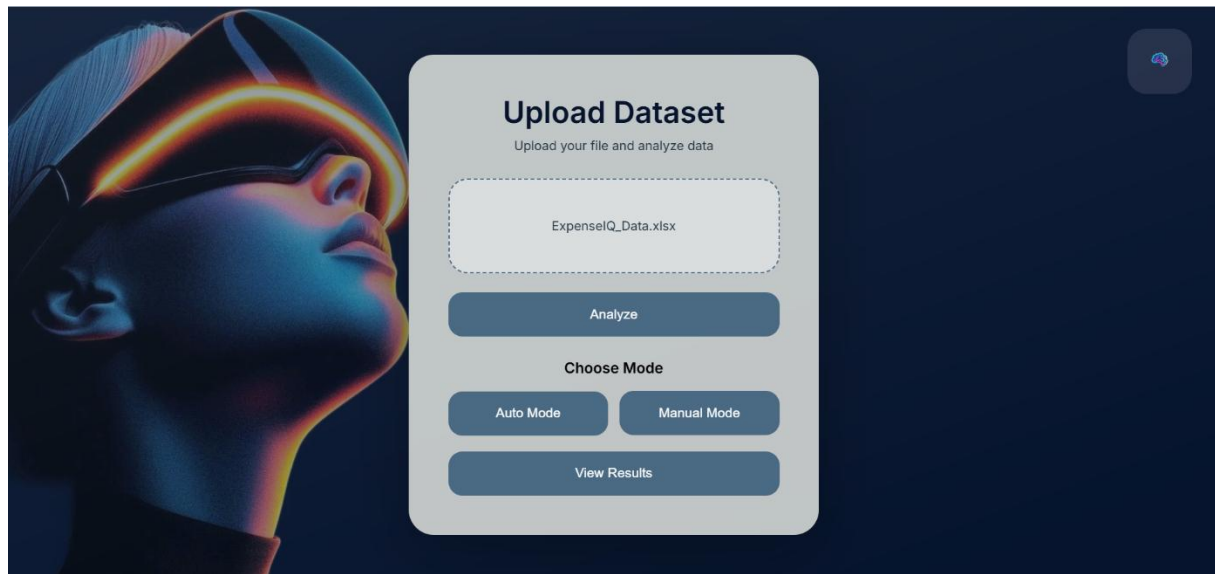


Figure 6.2.3

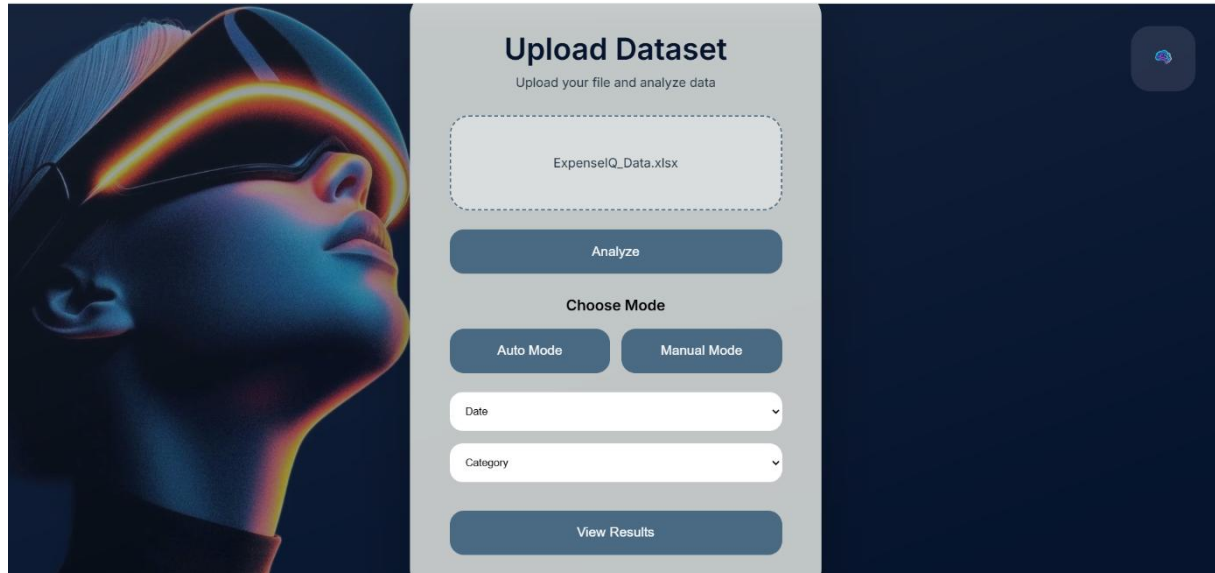


Figure 6.2.4

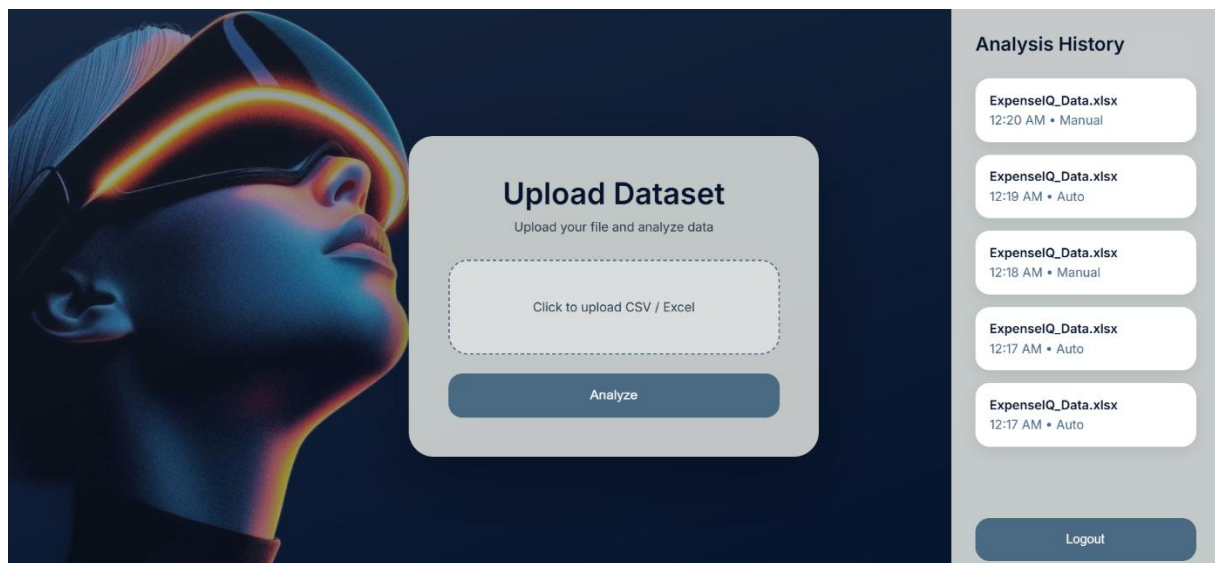


Figure 6.2.5

CHAPTER 7: RESULTS

The results of the **User-Centric Decision Intelligence System** demonstrate the successful processing of uploaded datasets and the generation of meaningful analytical outputs. The system effectively integrates data input, analysis, and visualization to support user decision-making.

7.1 System Output

The system performs the following operations successfully:

- Accepts dataset input from the user
- Stores uploaded data and maintains history
- Processes data based on user-selected options
- Generates analytical outputs
- Displays results in a structured format

7.2 Analysis Results

The system produces multiple types of outputs based on the dataset:

Graphical Output

- Charts and visual representations of data
- Helps in identifying trends and patterns

Statistical Output

- Summary statistics derived from the dataset
- Provides insights into data distribution

Insights and Recommendations

- Observations generated from analysis
- Helps users interpret results effectively



Figure 7.1: Analysis Output for Expense Dataset (User Integration Module)

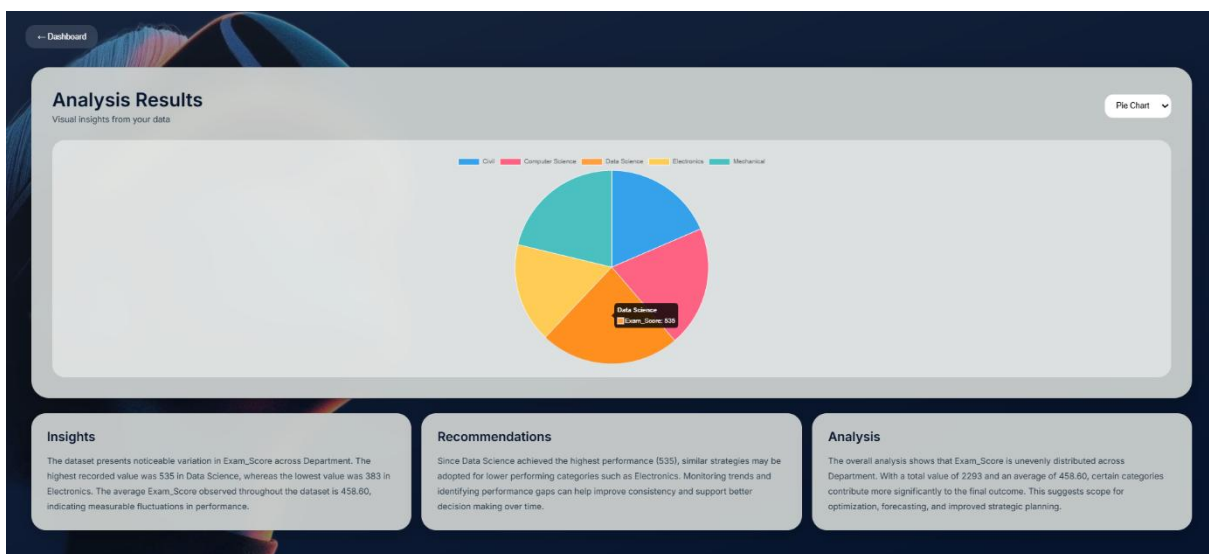


Figure 7.2: Analysis Output for General Dataset

CHAPTER 8: CONCLUSION

The User-Centric Decision Intelligence System has been successfully designed and implemented as an integrated platform that combines user interaction with data analytics. The system provides a seamless workflow starting from user authentication, dataset upload, and data processing, to the generation of meaningful insights and visual outputs.

One of the key strengths of the system lies in its ability to simplify complex data analysis tasks through an intuitive web-based interface. Users can upload datasets, choose analysis methods through automatic or manual selection, and obtain results in the form of charts, summaries, and recommendations without requiring advanced technical expertise. This makes the system accessible to a wider range of users.

The implementation also ensures effective data management through features such as dataset storage and history tracking. Users can revisit previously analyzed data, which enhances usability and provides continuity in decision-making processes. The system's modular design further supports flexibility and maintainability.

The results obtained from the system demonstrate its capability to handle different types of datasets, including user-specific data such as expense records and general datasets used for analysis. The generated outputs are accurate, visually clear, and informative, thereby supporting better understanding and interpretation of data.

In addition, the use of modern technologies such as Python for backend processing and HTML, CSS, and JavaScript for frontend development ensures efficient performance and smooth interaction between system components. The integration of these technologies contributes to a robust and scalable solution.

Overall, the project successfully achieves its objective of developing a user-centric platform for decision intelligence. It highlights the importance of integrating data handling, analysis, and visualization into a single system, making it a practical solution for real-world applications. The system can serve as a foundation for further enhancements and advanced analytical features in the future.



CHAPTER 9: FUTURE ENHANCEMENTS

The User-Centric Decision Intelligence System can be further enhanced to improve its scalability, intelligence, and real-world applicability. The following enhancements outline the future scope of the system.

I. Advanced User Management and Security

The system can be extended to support a multi-user environment where each user has a secure and personalized workspace. By implementing encrypted authentication mechanisms and secure session management, users will be able to safely store and access their individual datasets without interference from others.

II. Large-Scale Data Processing

The system can be enhanced to efficiently handle large and complex datasets. By incorporating optimized data processing techniques and improved memory management, the system will be capable of performing analysis on high-volume data with better performance.

III. Intelligent Data Selection

Future versions of the system can include fully automated data selection mechanisms.

The system can be designed to understand user requirements and automatically select relevant columns for analysis, including both numerical and textual data, eliminating the need for manual intervention.

IV. Integration of Machine Learning and NLP

The system can be upgraded by integrating Machine Learning (ML) and Natural Language Processing (NLP) techniques. This will enable:

- Predictive analysis
- Pattern recognition
- Generation of dynamic, human-like explanations

The system will be able to interpret user queries and provide more intelligent and context-aware insights.

V. Enhanced Data Visualization

The visualization capabilities of the system can be expanded to include advanced chart types such as scatter plots, histograms, and clustering-based visualizations. Interactive dashboards can also be introduced to allow users to explore data more effectively.

VI. Cloud Integration

The system can be deployed using modern cloud technologies to provide scalable storage and remote accessibility. Cloud integration will enable:

- Secure data storage
- Multi-user access
- Improved system availability

VII. Smart Recommendation Engine

The system can be enhanced to generate intelligent recommendations by analyzing patterns within the dataset. It can identify key trends, detect anomalies, and suggest possible solutions, making it a more proactive decision-support system.

VIII. Multi-Domain Data Support

The system can be extended to support a wide range of datasets such as:

- Financial data
- Sales and business data
- Transactional and operational data

This will increase the applicability of the system across different domains and industries.

9.1 Summary

The proposed enhancements aim to transform the system into a more intelligent, scalable, and user-centric platform. By incorporating advanced technologies and expanding system capabilities, the application can evolve into a comprehensive decision intelligence solution suitable for real-world applications.

CHAPTER 10: REFERENCES

The following resources were referred to during the development of the **User-Centric Decision Intelligence System**:

10.1 Books

1. Wes McKinney, *Python for Data Analysis*, O'Reilly Media. -This book was referred to for understanding data handling, cleaning, manipulation, and analysis using Python libraries such as Pandas and NumPy. It provided guidance for efficient dataset processing and analytical workflows.
2. Jake VanderPlas, *Python Data Science Handbook*, O'Reilly Media. - This book served as a reference for data science concepts, visualization methods, and analytical approaches used for extracting meaningful insights from datasets.

10.2 Websites

1. Python Official Documentation – <https://docs.python.org/>
2. Pandas Documentation – <https://pandas.pydata.org/docs/>
3. NumPy Documentation – <https://numpy.org/doc/>
4. MDN Web Docs – <https://developer.mozilla.org/>
5. W3Schools – <https://www.w3schools.com/>